

OOPS MINI PROJECT

REPORT

SHRIRAM GOPALAKRISHNAN 3122235001126

SURIYHA M 3122235001142

SURIYARAJ JP 3122235001141

1.PROBLEM STATEMENT:

STUDENT MANAGEMENT SYSTEM:

A portal where students can check their class timetable, submit their assignments, check their marks and attendance and teachers can update the student's attendance, assign assignments and update the marks.

2.MOTIVATION OF THE PROBLEM:

Managing academic tasks efficiently is essential for both students and teachers in educational institutions. Traditional methods of record-keeping and communication, such as paper-based systems or disparate digital tools, often lead to issues like lost information, delayed updates, and miscommunication. A Student Management System aims to resolve these issues by providing a centralized, accessible, and user-friendly portal for both students and teachers

3.SCOPE AND LIMITATIONS:

Scope

The Student Management System (SMS) is designed to provide essential tools and features that streamline academic management for students and teachers. The scope of the system includes:

1. Student Features:

- **Class Timetable Access:** Students can view their scheduled classes to plan their days efficiently.
- **Assignment Submission:** A digital space for students to submit assignments on time and track their submission status.
- **Grades and Performance Tracking:** Students can view their marks and monitor their academic performance across various subjects.

- **Attendance Tracking:** Provides access to attendance records, helping students stay informed about their attendance status and any necessary requirements to meet attendance criteria.

2. Teacher Features:

- **Attendance Management:** Teachers can mark and update attendance in real-time, which then becomes accessible to students.
- **Assignment Creation and Distribution:** Teachers can create, distribute, and track assignments for students, ensuring a streamlined assignment management process.
- **Grade Management:** Teachers can update and manage student marks directly within the system, reducing paperwork and enhancing grade transparency.

3. Admin Features (if applicable):

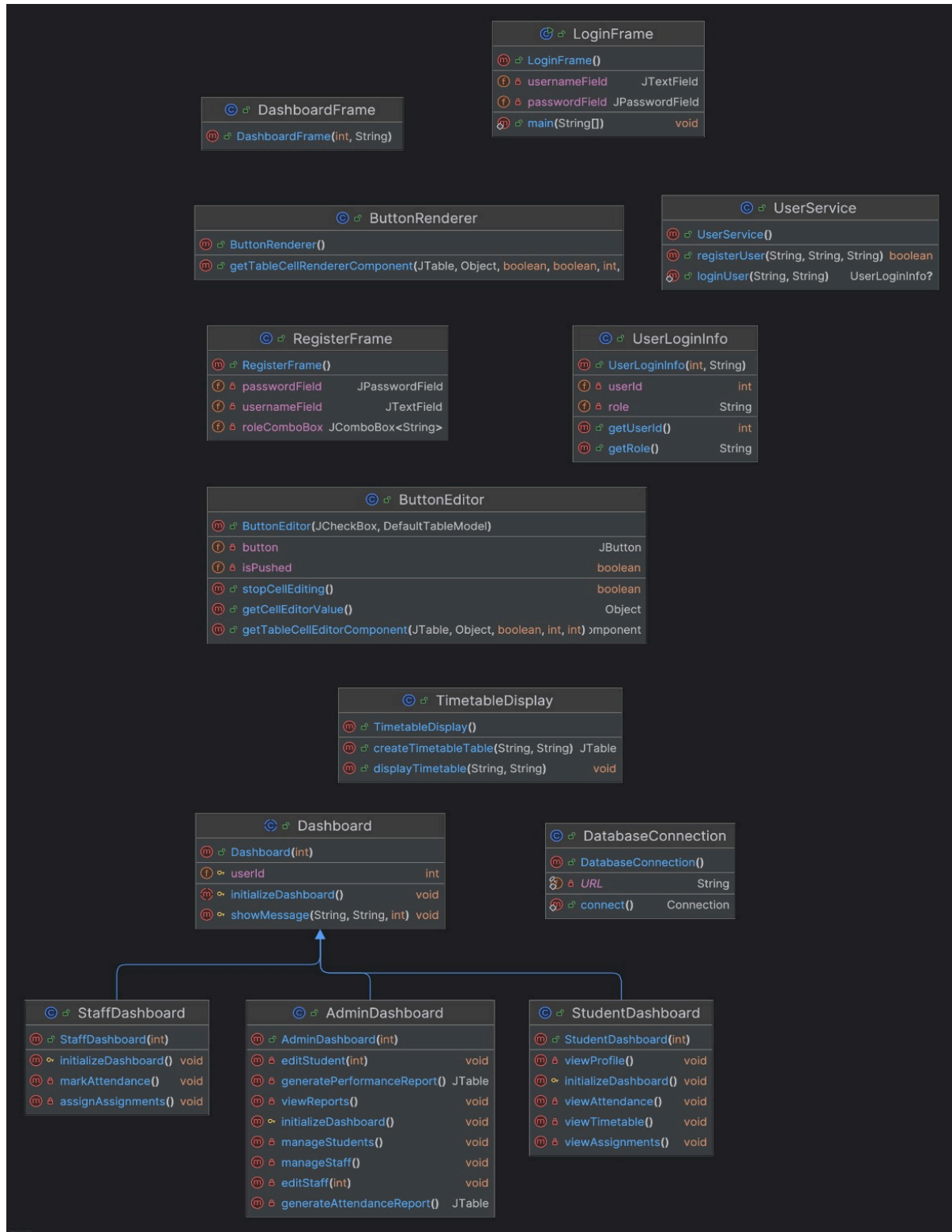
- **User Management:** Admin can add or remove users (students and teachers) and manage their access levels.
- **System Maintenance:** Admins oversee system updates, data backups, and other maintenance tasks to ensure smooth operation.

Limitations

Despite its comprehensive scope, the Student Management System may have some limitations:

1. **Limited Personalization:** The system may lack extensive personalization options for users, as it focuses on providing core academic features.
2. **Data Privacy Concerns:** Ensuring data privacy and security for sensitive information such as grades and attendance records may be challenging, especially if stringent security protocols are not in place.
3. **Scalability Constraints:** Depending on its initial design, the system might face scalability issues if expanded to accommodate a large number of users or institutions.
4. **User Adaptability:** Teachers and students with limited technical proficiency may find it challenging to use the system effectively without adequate training or support.
5. **Maintenance and Upkeep:** Regular maintenance and updates are required to keep the system secure and functional. A lack of proper maintenance could result in technical issues and hinder usability.

4.DESIGN OF THE SOLUTION (CLASS DIAGRAM):



5. MODULES SPLIT-UP:

1. UI (User Interface) Module

- Files: RegisterFrame.java, TimetableDisplay.java, etc.
- Description: Manages the user interfaces for different functionalities, such as registration and timetable display, enabling interaction between the system and users.

2. Database Module

- Files: DatabaseConnection.java
- Description: Handles database connections and operations, likely managing data storage and retrieval for students, assignments, attendance, etc.

3. Utility Module

- Files: ButtonEditor.java, ButtonRenderer.java
- Description: Contains utility classes that provide additional functionalities, such as customizing button displays and actions within the UI.

4. Dashboard Module

- Files: StudentDashboard.java
- Description: Provides a centralized interface for students, potentially aggregating information on attendance, grades, assignments, and more.

6.IMPLEMENTATION SPECIFICS:

1. UI (User Interface) Module

- Primary Classes: RegisterFrame.java, TimetableDisplay.java
- Responsibilities:
 - RegisterFrame.java:
 - Handles the user registration interface.
 - Likely contains forms for student or teacher data entry.
 - May include validations (e.g., input checks) and interactions with the database to save new user information.
 - TimetableDisplay.java:
 - Displays students' class schedules.
 - Possibly fetches data from the database and dynamically updates the UI.
 - May include filters for viewing the timetable by week, day, or subject.
- Implementation Notes:
 - The UI classes will use layouts and components (e.g., labels, buttons, tables) to structure the interface.
 - Event handling for interactions, such as button clicks, will be managed within these classes.

2. Database Module

- Primary Class: DatabaseConnection.java

- **Responsibilities:**
 - Manages the connection to the database, potentially using JDBC (Java Database Connectivity) if it's a Java-based project.
 - Contains methods for establishing, maintaining, and closing database connections.
 - Ensures that SQL queries and transactions are executed efficiently and securely.
 - May include helper functions for standard CRUD operations (Create, Read, Update, Delete) on student, teacher, attendance, and assignment records.
- **Implementation Notes:**
 - Ensure database credentials and sensitive information are securely managed.
 - Utilize prepared statements to prevent SQL injection attacks.

3. Utility Module

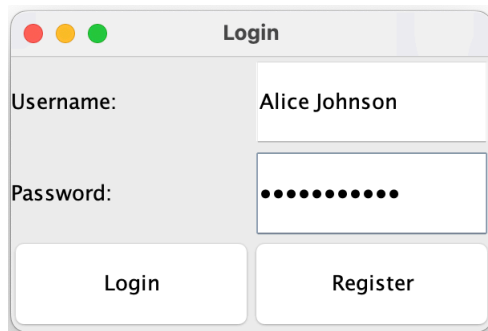
- **Primary Classes:** ButtonEditor.java, ButtonRenderer.java
- **Responsibilities:**
 - **ButtonEditor.java:**
 - Allows buttons within tables or lists to be edited. Useful if the UI contains interactive lists or grids.
 - Provides functionality for editing entries directly through buttons, like updating assignment statuses or attendance records.
 - **ButtonRenderer.java:**

- Manages the display and styling of buttons within components.
 - Ensures buttons are visually consistent and match the theme or style of the interface.
- Implementation Notes:
 - These utility classes will often interact with components from the UI module and may rely on event listeners for functionality.

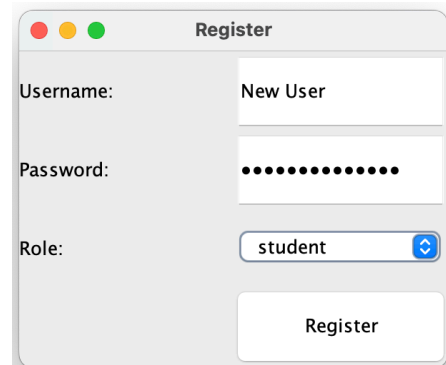
4. Dashboard Module

- Primary Class: StudentDashboard.java
- Responsibilities:
 - Acts as the main hub for student activities, consolidating access to different features (e.g., viewing grades, attendance, assignments).
 - Retrieves data from the database, such as marks, assignment deadlines, and attendance, and displays it in a unified interface.
 - May include graphical elements like charts or graphs to visualize performance metrics.
- Implementation Notes:
 - Ensure smooth data fetching and efficient loading of dashboard elements to enhance user experience.
 - Implement filtering and sorting options for better usability.

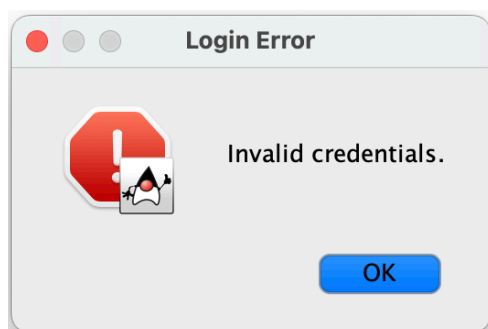
7.OUTPUT SCREENSHOTS:



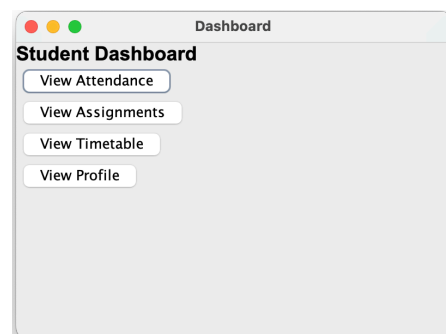
A screenshot of a 'Login' window. It has a title bar with three colored buttons (red, yellow, green) and the title 'Login'. The window contains two input fields: 'Username:' with the text 'Alice Johnson' and 'Password:' with a masked password '.....'. Below the fields are two buttons: 'Login' and 'Register'.



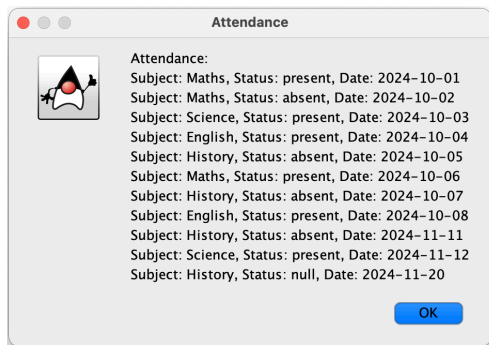
A screenshot of a 'Register' window. It has a title bar with three colored buttons (red, yellow, green) and the title 'Register'. The window contains three input fields: 'Username:' with the text 'New User', 'Password:' with a masked password '.....', and 'Role:' with a dropdown menu showing 'student'. Below the fields is a 'Register' button.



A screenshot of a 'Login Error' window. It has a title bar with three colored buttons (red, yellow, green) and the title 'Login Error'. The window contains a red octagonal stop sign icon with a small character inside, and the text 'Invalid credentials.' Below this is an 'OK' button.



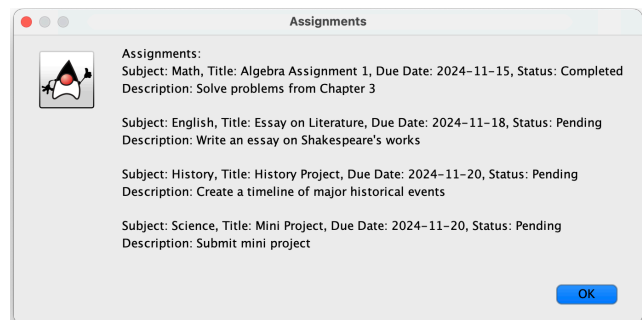
A screenshot of a 'Dashboard' window. It has a title bar with three colored buttons (red, yellow, green) and the title 'Dashboard'. The window contains the text 'Student Dashboard' and four buttons: 'View Attendance', 'View Assignments', 'View Timetable', and 'View Profile'.



A screenshot of an 'Attendance' window. It has a title bar with three colored buttons (red, yellow, green) and the title 'Attendance'. The window contains a small character icon and a list of attendance records. Below the list is an 'OK' button.

Attendance:

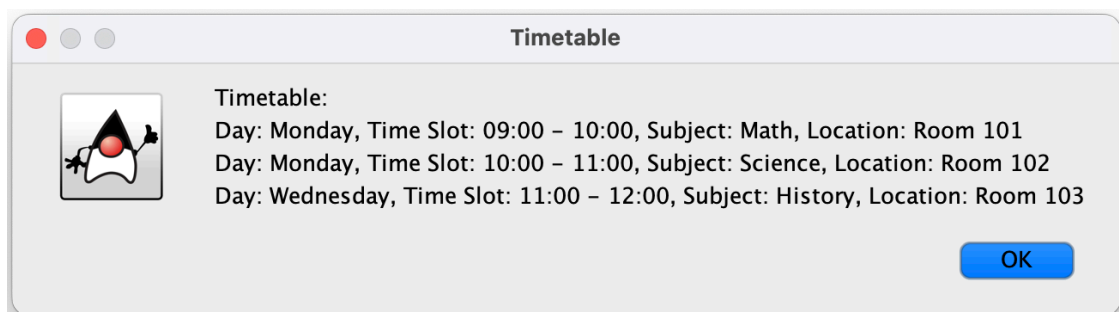
Subject: Maths, Status: present, Date: 2024-10-01
Subject: Maths, Status: absent, Date: 2024-10-02
Subject: Science, Status: present, Date: 2024-10-03
Subject: English, Status: present, Date: 2024-10-04
Subject: History, Status: absent, Date: 2024-10-05
Subject: Maths, Status: present, Date: 2024-10-06
Subject: History, Status: absent, Date: 2024-10-07
Subject: English, Status: present, Date: 2024-10-08
Subject: History, Status: absent, Date: 2024-11-11
Subject: Science, Status: present, Date: 2024-11-12
Subject: History, Status: null, Date: 2024-11-20



A screenshot of an 'Assignments' window. It has a title bar with three colored buttons (red, yellow, green) and the title 'Assignments'. The window contains a small character icon and a list of assignments. Below the list is an 'OK' button.

Assignments:

Subject: Math, Title: Algebra Assignment 1, Due Date: 2024-11-15, Status: Completed
Description: Solve problems from Chapter 3
Subject: English, Title: Essay on Literature, Due Date: 2024-11-18, Status: Pending
Description: Write an essay on Shakespeare's works
Subject: History, Title: History Project, Due Date: 2024-11-20, Status: Pending
Description: Create a timeline of major historical events
Subject: Science, Title: Mini Project, Due Date: 2024-11-20, Status: Pending
Description: Submit mini project



A screenshot of a 'Timetable' window. It has a title bar with three colored buttons (red, yellow, green) and the title 'Timetable'. The window contains a small character icon and a list of timetable entries. Below the list is an 'OK' button.

Timetable:

Day: Monday, Time Slot: 09:00 – 10:00, Subject: Math, Location: Room 101
Day: Monday, Time Slot: 10:00 – 11:00, Subject: Science, Location: Room 102
Day: Wednesday, Time Slot: 11:00 – 12:00, Subject: History, Location: Room 103

Student Profile



Username: Alice Johnson
Class: 10
Section: A

OK

Dashboard

Staff Dashboard

Mark Attendance

Assign Assignments

Input



Enter Class:

10

Cancel OK

Input



Enter Section:

A

Cancel OK

Input



Enter Date (YYYY-MM-DD):

Cancel OK

Input



Mark attendance for Student ID 69 (present/absent):

present

Cancel OK

Success



Attendance marked successfully.

OK

Input



Enter Assignment Title:

Cancel OK

Input



Enter Description:

Cancel OK

Input



Enter Due Date (YYYY-MM-DD):

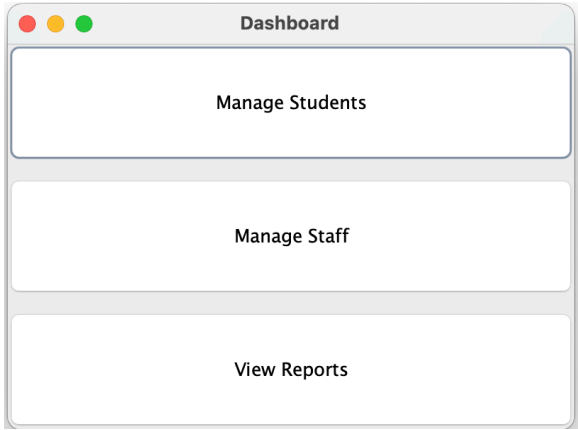
Cancel OK

Success



Assignment assigned successfully to the entire class and section.

OK



A window titled "Manage Students" displaying a table with 5 columns: Student ID, Username, Class, Section, and Action. The table contains 16 rows of student data, each with an "Edit" button in the Action column.

Student ID	Username	Class	Section	Action
69	Alice Johnson	10	A	Edit
70	Bob Smith	10	A	Edit
71	Charlie Brown	10	A	Edit
72	Daisy Lee	10	A	Edit
73	Ethan White	10	A	Edit
74	Fiona Black	10	A	Edit
75	George Green	10	A	Edit
76	Hannah King	10	A	Edit
77	Ivy Adams	10	A	Edit
78	Jack Wilson	10	A	Edit
79	Liam Brown	10	B	Edit
80	Mia Thompson	10	B	Edit
81	Noah Taylor	10	B	Edit
82	Olivia Anderson	10	B	Edit
83	Peter Thomas	10	B	Edit
84	Quinn Harris	10	B	Edit

A window titled "Manage Staff" displaying a table with 3 columns: Staff ID, Username, and Action. The table contains 6 rows of staff data, each with an "Edit" button in the Action column.

Staff ID	Username	Action
1001	Alice Johnson	Edit
1002	Robert Miller	Edit
1003	Catherine Davis	Edit
1004	David Wilson	Edit
1005	Eva Brown	Edit
1006	Michael Taylor	Edit

A window titled "Reports" with two tabs: "Attendance Report" and "Performance Report". The "Attendance Report" tab is active, showing a table with 4 columns: Student ID, Class, Section, and Attendance Status. The table contains 16 rows of attendance data.

Student ID	Class	Section	Attendance Status
69	10	A	present
69	10	A	absent
69	10	A	present
69	10	A	present
69	10	A	absent
94	11	A	present
94	11	A	absent
94	11	A	present
94	11	A	present
94	11	A	absent
88	10	B	absent
88	10	B	present
88	10	B	absent

A window titled "Reports" with two tabs: "Attendance Report" and "Performance Report". The "Performance Report" tab is active, showing a form with 5 input fields: Student ID, Name, Subject, Completed Assign..., and Grade.

Student ID	Name	Subject	Completed Assign...	Grade
------------	------	---------	---------------------	-------

8.OBJECT ORIENTED FEATURES USED:

1. Encapsulation

- Purpose: Encapsulation is used to restrict direct access to class data and control how it is modified.
- Examples:
 - Classes like RegisterFrame and TimetableDisplay encapsulate user interface logic, ensuring that UI components and data are kept private to those classes.
 - DatabaseConnection encapsulates the database connection details, providing methods to open and close connections while keeping sensitive data like credentials secure.
- Benefits: Protects data integrity and provides controlled access to class data through getter and setter methods.

2. Abstraction

- Purpose: Abstraction simplifies complex processes by hiding unnecessary details and showing only relevant information to the user.
- Examples:
 - DatabaseConnection abstracts the database operations, exposing only essential methods like connect, disconnect, and executeQuery, rather than the intricate details of SQL connection handling.
 - Utility classes like ButtonRenderer abstract the rendering and editing functionalities of buttons within tables, hiding the specifics of how buttons are displayed or edited.

- **Benefits:** Simplifies usage for developers by providing a clear interface to interact with, hiding complex implementation details.

3. Inheritance

- **Purpose:** Inheritance allows a class to inherit attributes and methods from another class, enabling code reuse and a hierarchical organization.
- **Examples:**
 - Classes for UI components (e.g., RegisterFrame, TimetableDisplay) may inherit from a common JFrame or JPanel superclass, allowing them to leverage base functionality for graphical elements.
 - If there are commonalities between different types of dashboards (e.g., StudentDashboard and potentially TeacherDashboard), inheritance may be used to create a generic Dashboard class that can be extended.
- **Benefits:** Encourages code reuse and supports a hierarchical structure, making it easier to manage and extend the system.

4. Polymorphism

- **Purpose:** Polymorphism allows the same operation to behave differently on different classes.
- **Examples:**
 - **Method Overriding:** Subclasses like StudentDashboard could override methods of a general Dashboard class to display specific information, such as student-specific data versus teacher-specific data.

- Interfaces: UI components or database operations might implement interfaces that allow them to have polymorphic behavior, where different implementations provide specific functionality (e.g., different displays for students and teachers).
- Benefits: Provides flexibility, allowing the system to adapt to different contexts without changing the underlying code

5.Collections

- Vector (in TimetableDisplay.java):
 - Used to manage rows in tables, likely through DefaultTableModel, which internally uses Vector for handling data in JTable.
- DefaultTableModel (in TimetableDisplay.java, ButtonEditor.java, and AdminDashboard.java):
 - Utilized for managing data displayed in JTable components. It supports dynamic table creation and modification, often used to represent complex, tabular data structures in the user interface.
- ArrayList (in StaffDashboard.java):
 - Likely used to store lists of items dynamically, such as staff records or other elements that require ordered lists with flexible sizes.

9. INFERENCE AND FUTURE EXTENSION:

Inference:

This college student management application provides a streamlined approach for managing student, staff, and admin activities within an educational institution. By integrating attendance tracking, assignment management, and timetable viewing, the application centralizes critical information and facilitates efficient workflows for students, faculty, and administration. The modular design, with separate dashboards for each role, ensures a tailored user experience and makes maintenance and future development easier.

Future Extensions:

- **Advanced Analytics and Reporting:** Implement features to analyze attendance trends, assignment completion rates, and overall academic performance, providing actionable insights to faculty and administrators.
- **Automated Notifications and Reminders:** Add push notifications or email reminders for assignment deadlines, attendance issues, and upcoming classes or exams to improve student engagement and accountability.
- **Mobile Application Integration:** Develop a mobile-friendly version to enhance accessibility, allowing students and staff to interact with the system on the go.
- **Role-Based Access Controls:** Expand user roles and permissions for improved security, limiting access to sensitive information based on the user's role in the institution.

- **Integration with Learning Management Systems (LMS):**
Enable seamless data sharing with popular LMS platforms to streamline information exchange, such as grades and course materials, between the systems.

These enhancements would significantly expand the system's utility, providing greater value to educational institutions aiming for efficient and data-driven management.